

Diseño POO Auto UML

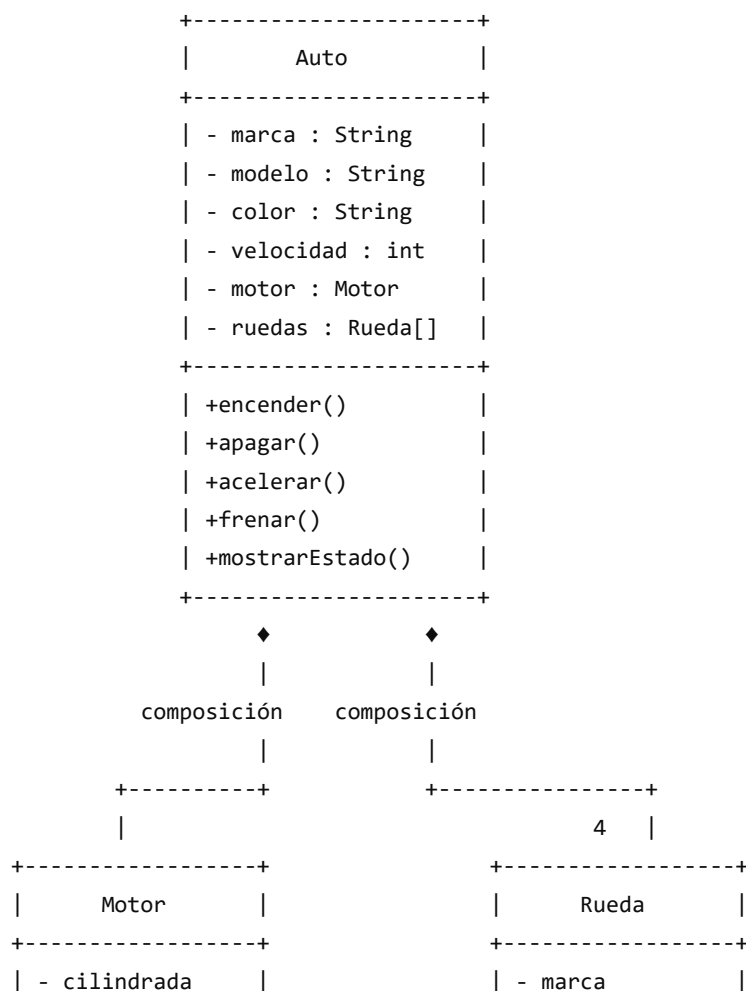
Vamos a diseñar la clase "Auto" con atributos y métodos para objetos de la clase Auto. Vamos a implementar relación entre las clases Motor y Rueda. También plantearemos atributos y eventualmente métodos para estas clases también. Finalmente vamos a devolver el código para realizar el diagrama UML correspondiente.

Este ejemplo es muy bueno para introducir **POO**, ya que permite mostrar claramente los distintos tipos de relaciones entre objetos.

En este caso podemos plantear:

- **Auto** → tiene un **Motor** (**composición**: el motor pertenece al auto).
- **Auto** → tiene **4 Ruedas** (**composición**: las ruedas forman parte del auto).
- Cada clase posee sus propios atributos y métodos.

Diagrama UML propuesto



```

| - potencia      |
| - encendido     |
+-----+
| +encender()     |
| +apagar()       |
| +mostrarEstado() |
+-----+

```

```

| - rodado        |
| - presion       |
+-----+
| +inflar()       |
| +desinflar()    |
| +mostrarEstado() |
+-----+

```

Clase Motor

</> Java

```

public class Motor {

    private int cilindrada;
    private int potencia;
    private boolean encendido;

    public Motor(int cilindrada, int potencia) {
        this.cilindrada = cilindrada;
        this.potencia = potencia;
        this.encendido = false;
    }

    public void encender() {
        encendido = true;
        System.out.println("Motor encendido");
    }

    public void apagar() {
        encendido = false;
        System.out.println("Motor apagado");
    }

    public void mostrarEstado() {
        System.out.println("Motor " + cilindrada + " cc");
        System.out.println("Potencia: " + potencia + " HP");
        System.out.println("Encendido: " + encendido);
    }

    public boolean isEncendido() {
        return encendido;
    }

}

```

Clase Rueda

</> Java

```
public class Rueda {

    private String marca;
    private int rodado;
    private double presion;

    public Rueda(String marca, int rodado) {
        this.marca = marca;
        this.rodado = rodado;
        this.presion = 32;
    }

    public void inflar() {
        presion = 32;
    }

    public void desinflar() {
        presion = 15;
    }

    public void mostrarEstado() {
        System.out.println("Marca: " + marca);
        System.out.println("Rodado: " + rodado);
        System.out.println("Presión: " + presion + " PSI");
    }
}
```

Clase Auto

</> Java

```
public class Auto {

    private String marca;
    private String modelo;
    private String color;
    private int velocidad;

    private Motor motor;
    private Rueda[] ruedas;

    public Auto(String marca,
                String modelo,
                String color,
                Motor motor,
                Rueda[] ruedas) {

        this.marca = marca;
        this.modelo = modelo;
```

```
        this.color = color;
        this.motor = motor;
        this.ruedas = ruedas;
        this.velocidad = 0;
    }

    public void encender() {
        motor.encender();
    }

    public void apagar() {
        motor.apagar();
        velocidad = 0;
    }

    public void acelerar(int incremento) {

        if (motor.isEncendido()) {
            velocidad += incremento;
            System.out.println("Velocidad: " + velocidad + " km/h");
        } else {
            System.out.println("No se puede acelerar. El motor está apagado.");
        }

    }

    public void frenar(int decremento) {

        velocidad -= decremento;

        if (velocidad < 0) {
            velocidad = 0;
        }

        System.out.println("Velocidad: " + velocidad + " km/h");
    }

    public void mostrarEstado() {

        System.out.println("Marca: " + marca);
        System.out.println("Modelo: " + modelo);
        System.out.println("Color: " + color);
        System.out.println("Velocidad: " + velocidad);

        motor.mostrarEstado();

        System.out.println("Cantidad de ruedas: " + ruedas.length);
    }
}
```

Clase Principal

</> Java

```
public class App {  
  
    public static void main(String[] args) {  
  
        Motor motor = new Motor(2000, 150);  
  
        Rueda[] ruedas = new Rueda[4];  
  
        for (int i = 0; i < ruedas.length; i++) {  
            ruedas[i] = new Rueda("Pirelli", 17);  
        }  
  
        Auto auto = new Auto(  
            "Toyota",  
            "Corolla",  
            "Blanco",  
            motor,  
            ruedas  
        );  
  
        auto.mostrarEstado();  
  
        auto.encender();  
  
        auto.acelerar(30);  
  
        auto.acelerar(40);  
  
        auto.frenar(20);  
  
        auto.apagar();  
  
    }  
  
}
```

Relaciones UML

Este ejemplo permite explicar muy bien las relaciones entre objetos:

Relación	Clases	Tipo UML	Explicación
Auto → Motor	1 a 1	Composición (◆)	El motor forma parte del auto. Si el auto deja de existir, su motor también dentro del modelo.

Relación	Clases	Tipo UML	Explicación
Auto → Rueda	1 a 4	Composición (◆)	Las cuatro ruedas pertenecen al auto.

En UML, estas relaciones podrían representarse así:

```

Auto ◆----- 1 Motor
Auto ◆----- 4 Rueda

```

Donde el **rombo negro (◆)** indica **composición**, es decir, una relación de "todo-parte" con fuerte dependencia del ciclo de vida. Este ejemplo es especialmente adecuado para introducir la diferencia entre **composición** y **agregación**, ya que posteriormente puede ampliarse con una clase `Conductor`, donde el conductor y el auto existirían de forma independiente y la relación sería de **agregación**.



Algunos cambios... el auto tiene cinco ruedas (incluir el auxilio), y en el diagrama de UML incluir la cardinalidad. El diagrama es para desplegar en Mermaid

Me parece mejor aún. La rueda de auxilio hace que el ejemplo sea más realista y, además, permite explicar la **cardinalidad exacta** de la relación.

En Mermaid, el diagrama UML quedaría así:



Mermaid

```

classDiagram
class Auto{
    -String marca
    -String modelo
    -String color
    -int velocidad
    -Motor motor
    -Rueda[] ruedas

    +Auto(marca, modelo, color, motor, ruedas)
    +encender()
    +apagar()
    +acelerar(incremento)
    +frenar(decremento)
    +mostrarEstado()
}

```

Cardinalidades

Relación	Cardinalidad	Tipo
Auto → Motor	1..1	Composición
Auto → Rueda	1..5	Agregación

O dicho de otra forma:

- Un **Auto** posee exactamente **1 Motor**.
- Un **Motor** pertenece a un único **Auto** (en este modelo).
- Un **Auto** utiliza exactamente **5 Ruedas** (4 de rodado + 1 de auxilio).
- Una **Rueda** puede existir independientemente del **Auto**, por eso la relación es de agregación.

También modificaría el atributo del Auto

Java

```
private Rueda[] ruedas; // 5 ruedas (4 + auxilio)
```

y en la aplicación:

Java

```
Rueda[] ruedas = {  
    new Rueda("Pirelli",17),  
    new Rueda("Pirelli",17),  
    new Rueda("Pirelli",17),  
    new Rueda("Pirelli",17),  
    new Rueda("Pirelli",17) // rueda de auxilio  
};
```

Una sugerencia didáctica

Ya que este ejemplo está pensado para explicar relaciones entre objetos, haría una pequeña mejora adicional: reemplazar el arreglo por una colección.

Java

```
private ArrayList<Rueda> ruedas;
```

De esa forma, además de enseñar **agregación**, aprovechás para introducir el uso de colecciones (`ArrayList`), agregando las cinco ruedas con `add()` , lo que resulta más natural en POO que trabajar con un arreglo fijo. Luego, más adelante en el curso, incluso podrías reemplazar `ArrayList` por un `HashSet` o `HashMap` para reutilizar el mismo ejemplo al explicar colecciones.



si, la idea era implementarlo con colecciones, entonces devolveme los códigos de las clases actualizados

Sí, con colecciones queda más flexible y más orientado a objetos. Te dejo las clases actualizadas usando `ArrayList<Rueda>`.

Clase Motor

```
</> Java
public class Motor {

    private int cilindrada;
    private int potencia;
    private boolean encendido;

    public Motor(int cilindrada, int potencia) {
        this.cilindrada = cilindrada;
        this.potencia = potencia;
        this.encendido = false;
    }

    public void encender() {
        encendido = true;
        System.out.println("Motor encendido");
    }

    public void apagar() {
        encendido = false;
        System.out.println("Motor apagado");
    }

    public void mostrarEstado() {
        System.out.println("Motor " + cilindrada + " cc");
        System.out.println("Potencia: " + potencia + " HP");
        System.out.println("Encendido: " + encendido);
    }

    public boolean isEncendido() {
        return encendido;
    }

}
```

Clase Rueda

```
</> Java
public class Rueda {
```

```
private String marca;
private int rodado;
private double presion;
private boolean auxilio;

public Rueda(String marca, int rodado, boolean auxilio) {
    this.marca = marca;
    this.rodado = rodado;
    this.auxilio = auxilio;
    this.presion = 32;
}

public void inflar() {
    presion = 32;
    System.out.println("Rueda inflada");
}

public void desinflar() {
    presion = 15;
    System.out.println("Rueda desinflada");
}

public void mostrarEstado() {
    System.out.println("Marca: " + marca);
    System.out.println("Rodado: " + rodado);
    System.out.println("Presión: " + presion + " PSI");
    System.out.println("Auxilio: " + auxilio);
}
}
```

Clase Auto

<> Java

```
import java.util.ArrayList;

public class Auto {

    private String marca;
    private String modelo;
    private String color;
    private int velocidad;

    private Motor motor;
    private ArrayList<Rueda> ruedas;

    public Auto(String marca, String modelo, String color, Motor motor) {
        this.marca = marca;
        this.modelo = modelo;
        this.color = color;
        this.motor = motor;
        this.velocidad = 0;
        this.ruedas = new ArrayList<Rueda>();
    }
}
```

```
}

public void agregarRueda(Rueda rueda) {
    if (ruedas.size() < 5) {
        ruedas.add(rueda);
    } else {
        System.out.println("El auto ya tiene 5 ruedas.");
    }
}

public void encender() {
    motor.encender();
}

public void apagar() {
    motor.apagar();
    velocidad = 0;
}

public void acelerar(int incremento) {
    if (motor.isEncendido()) {
        velocidad += incremento;
        System.out.println("Velocidad actual: " + velocidad + " km/h");
    } else {
        System.out.println("No se puede acelerar. El motor está apagado.");
    }
}

public void frenar(int decremento) {
    velocidad -= decremento;

    if (velocidad < 0) {
        velocidad = 0;
    }

    System.out.println("Velocidad actual: " + velocidad + " km/h");
}

public void mostrarEstado() {
    System.out.println("Auto: " + marca + " " + modelo);
    System.out.println("Color: " + color);
    System.out.println("Velocidad: " + velocidad + " km/h");

    System.out.println("Estado del motor:");
    motor.mostrarEstado();

    System.out.println("Ruedas del auto:");
    for (Rueda rueda : ruedas) {
        rueda.mostrarEstado();
        System.out.println("-----");
    }
}
```

```

    }
}

```

Clase App

Java

```

public class App {

    public static void main(String[] args) {

        Motor motor = new Motor(2000, 150);

        Auto auto = new Auto(
            "Toyota",
            "Corolla",
            "Blanco",
            motor
        );

        auto.agregarRueda(new Rueda("Pirelli", 17, false));
        auto.agregarRueda(new Rueda("Pirelli", 17, false));
        auto.agregarRueda(new Rueda("Pirelli", 17, false));
        auto.agregarRueda(new Rueda("Pirelli", 17, false));
        auto.agregarRueda(new Rueda("Pirelli", 17, true));

        auto.mostrarEstado();

        auto.encender();
        auto.acelerar(30);
        auto.acelerar(40);
        auto.frenar(20);
        auto.apagar();

        System.out.println("Estado final:");
        auto.mostrarEstado();
    }
}

```

En el UML de Mermaid, el atributo quedaría así:

Java

```
-ArrayList~Rueda~ ruedas
```

Y la relación:

Mermaid

Diagrama inválido o no compatible.



Definir la clase Persona en Python con atributos y metodos, incluyendo el constructor de Persona

Un buen primer ejemplo en Python es una clase `Persona` sencilla, donde se puedan explicar claramente los conceptos de **atributos**, **métodos**, **constructor** y uso de `self`.

Python

```
class Persona:

    # Constructor
    def __init__(self, nombre, apellido, edad):
        self.nombre = nombre
        self.apellido = apellido
        self.edad = edad

    # Método para saludar
    def saludar(self):
        print(f"Hola, soy {self.nombre} {self.apellido}.")

    # Método para cumplir años
    def cumplir_anios(self):
        self.edad += 1
        print(f"Ahora tengo {self.edad} años.")

    # Método para mostrar los datos
    def mostrar_datos(self):
        print("Nombre:", self.nombre)
        print("Apellido:", self.apellido)
        print("Edad:", self.edad)
```

Programa principal

Python

```
# Crear un objeto Persona
persona1 = Persona("Juan", "Pérez", 25)

# Invocar métodos
persona1.saludar()
persona1.mostrar_datos()

persona1.cumplir_anios()
```

```
persona1.mostrar_datos()
```

Salida esperada

Hola, soy Juan Pérez.

Nombre: Juan

Apellido: Pérez

Edad: 25

Ahora tengo 26 años.

Nombre: Juan

Apellido: Pérez

Edad: 26

Explicación de cada parte

Atributos

Son las características del objeto.

Python

```
self.nombre  
self.apellido  
self.edad
```

Cada objeto `Persona` tendrá sus propios valores para estos atributos.

Constructor

En Python el constructor se llama `__init__`.

Python

```
def __init__(self, nombre, apellido, edad):
```

Se ejecuta automáticamente cuando se crea un objeto.

Python

```
persona1 = Persona("Juan", "Pérez", 25)
```

Al crear el objeto, Python invoca automáticamente el constructor e inicializa los atributos.

self

`self` representa al objeto que está ejecutando el método.

Por ejemplo:

Python

```
self.nombre = nombre
```

significa:

"Guardar el valor recibido en el atributo `nombre` de este objeto."

Cada objeto tiene su propio `self`.

Métodos

Son las acciones que puede realizar un objeto.

Python

```
saludar()  
cumplir_anios()  
mostrar_datos()
```

Se invocan mediante la notación punto:

Python

```
persona1.saludar()  
persona1.cumplir_anios()  
persona1.mostrar_datos()
```

Diagrama UML

```
+-----+  
|      Persona      |  
+-----+  
| - nombre : str    |  
| - apellido : str  |  
| - edad : int      |  
+-----+  
| + __init__(...)   |  
| + saludar()       |  
| + cumplir_anios() |
```

```
| + mostrar_datos() |
+-----+
```

Este ejemplo es una excelente introducción a la POO en Python, ya que permite explicar de manera progresiva los conceptos fundamentales: **clase**, **objeto**, **atributos**, **constructor** (`__init__`), `self` y **métodos**, estableciendo un paralelismo muy claro con lenguajes como Java y C#.



vamos a generar las dos clases en python y la explicacion para ejecutar el programa

Este es el ejemplo que suelo recomendar para una primera clase de POO en Python. Mantiene la misma estructura que en Java o C#, con una clase `Persona` y una clase principal que crea el objeto y utiliza sus métodos.

Clase `Persona.py`

`<>` Python

```
class Persona:

    # Constructor
    def __init__(self, nombre, apellido, edad):
        self.nombre = nombre
        self.apellido = apellido
        self.edad = edad

    # Método para saludar
    def saludar(self):
        print(f"Hola, soy {self.nombre} {self.apellido}.")

    # Método para cumplir años
    def cumplir_anios(self):
        self.edad += 1
        print(f"¡Ahora tengo {self.edad} años!")

    # Método para mostrar Los datos
    def mostrar_datos(self):
        print("==== DATOS DE LA PERSONA =====")
        print(f"Nombre : {self.nombre}")
        print(f"Apellido : {self.apellido}")
        print(f"Edad : {self.edad}")
```

Clase Principal.py

Python

```
from Persona import Persona

# Crear un objeto Persona
persona1 = Persona("Juan", "Pérez", 25)

# Invocar métodos
persona1.saludar()

print()

persona1.mostrar_datos()

print()

persona1.cumplir_anios()

print()

persona1.mostrar_datos()
```

Estructura del proyecto

```
ProyectoPython/
|
├─ Persona.py
└─ Principal.py
```

Explicación del programa

1. Crear la clase

La instrucción

Python

```
class Persona:
```

define una nueva clase llamada **Persona**.

Una clase funciona como un plano o plantilla para crear objetos.

2. El constructor

En Python el constructor se llama `__init__`.

Python

```
def __init__(self, nombre, apellido, edad):
```

Se ejecuta automáticamente cuando se crea un objeto.

Dentro del constructor se inicializan los atributos.

Python

```
self.nombre = nombre  
self.apellido = apellido  
self.edad = edad
```

3. ¿Qué significa `self`?

`self` representa al objeto actual.

Cuando escribimos

Python

```
self.nombre
```

nos referimos al atributo **nombre** del objeto que está ejecutando el método.

Es equivalente al `this` de Java o C#.

4. Métodos

Los métodos representan las acciones que puede realizar un objeto.

Python

```
saludar()  
cumplir_anios()  
mostrar_datos()
```

Todos reciben como primer parámetro el objeto (`self`).

5. Crear un objeto

En la clase principal escribimos

</> Python

```
persona1 = Persona("Juan", "Pérez", 25)
```

Aquí ocurre lo siguiente:

- Se reserva memoria.
- Se crea un objeto.
- Se ejecuta automáticamente el constructor.
- La variable `persona1` referencia ese objeto.

6. Invocar métodos

Luego llamamos a los métodos mediante la notación punto.

</> Python

```
persona1.saludar()  
persona1.mostrar_datos()  
persona1.cumplir_anios()
```

Cada método actúa sobre el objeto `persona1`.

Salida esperada

```
Hola, soy Juan Pérez.
```

```
===== DATOS DE LA PERSONA =====
```

```
Nombre    : Juan
```

```
Apellido  : Pérez
```

```
Edad      : 25
```

```
¡Ahora tengo 26 años!
```

```
===== DATOS DE LA PERSONA =====
```

```
Nombre    : Juan
```

```
Apellido  : Pérez
```

```
Edad      : 26
```

¿Cómo ejecutar el programa?

Opción 1: Desde Visual Studio Code

1. Crear una carpeta llamada **ProyectoPython**.
2. Abrir esa carpeta en Visual Studio Code (**Archivo** → **Abrir carpeta**).

3. Crear los archivos:

- `Persona.py`
- `Principal.py`

4. Copiar el código correspondiente en cada archivo.

5. Verificar que Python esté instalado. En una terminal ejecutar:

</> Bash

```
python --version
```

o bien

</> Bash

```
python3 --version
```

6. Abrir el archivo `Principal.py`.

7. Presionar el botón **Ejecutar** de Visual Studio Code, o abrir una terminal integrada y ejecutar:

</> Bash

```
python Principal.py
```

En algunos sistemas puede ser necesario usar:

</> Bash

```
python3 Principal.py
```

Opción 2: Desde la terminal

Ubicarse en la carpeta del proyecto:

</> Bash

```
cd ProyectoPython
```

y ejecutar:

</> Bash

```
python Principal.py
```

Conceptos de POO que se trabajan con este ejemplo

-  Clase

- ✓ Objeto
- ✓ Atributos
- ✓ Constructor (`__init__`)
- ✓ Métodos
- ✓ Encapsulamiento básico mediante atributos del objeto
- ✓ Uso de `self`
- ✓ Creación de objetos
- ✓ Invocación de métodos
- ✓ Separación del programa en múltiples módulos (`Persona.py` y `Principal.py`)

Este ejemplo resulta especialmente útil para alumnos que ya conocen Java o C#, ya que les permite identificar rápidamente las equivalencias entre `class` , `this` y el constructor en esos lenguajes con `class` , `self` y `__init__` en Python.

